

TEAM MEMBERS

1.Palleti.Nagalakshmi(25A31A0587)

2.Paddada.Sai Tejeswari(25A31A0591)

3.Shaik.Ahmad Alisha(25A31A05AZ)

4.Tadi.Dharani Suresh(25A31A05BA)

Under Guidance

Mrs.T.N.V.Durga

Pragati Engineering College(A),Surampalem

PROJECT NAME:

Personal Dairy Applications

Description:

The Personal Diary Application is a GUI-based desktop software project developed in C++ that lets users write, save, edit, and view daily diary entries along with their dates. It is designed as a simple but practical personal record system where each entry is stored safely and can be retrieved later. The project is suitable for demonstrating event-driven programming, file handling, and modular software design in C++.

The main purpose of this application is to provide users with a digital diary that is easier to use than paper notes and more organized than plain text files. The application allows users to enter diary content through a graphical interface, attach the current or selected date, and store the information permanently. It can also support viewing past entries, updating entries, and deleting unwanted records. A GUI framework such as Qt is commonly used for this type of C++ desktop application because it provides ready-made widgets, windows, buttons, and text input controls.

- Write new diary entries with dates.
- Save entries locally for later access.
- View previously written diary records.
- Edit or delete selected entries.
- Keep the interface simple and user-friendly.

The application can be divided into four main parts: the **Presentation Layer** for the GUI, the **Control Layer** for handling user actions, the **Business Logic Layer** for diary operations, and the **Data Storage Layer** for saving and loading records. The GUI captures input from the user, the control layer sends commands to the diary manager, the logic layer processes the entry data, and the storage layer writes the information to a file or database. This separation makes the code easier to understand, test, and extend.

Each diary record typically contains a date, optional title, and the main diary text. When the user clicks Save, the application validates the input and stores the entry in a structured format such as a text file, JSON file, or SQLite database. When the user opens an old entry, the program reads the stored data and displays it back in the GUI. This approach ensures that diary content remains available after the application is closed and reopened.

System Architecture:

The Personal Diary Application is a **GUI-based desktop application** developed in C++ that allows users to create, edit, view, and save daily diary entries with dates. The system is designed with a modular structure so that the user interface, diary management logic, and data storage components remain independent and easy to maintain.

Objectives

- Provide a simple graphical interface for writing personal diary entries.
- Save diary records with associated dates.
- Allow users to view, edit, and manage past entries.
- Ensure a structured design suitable for future enhancement.

High-Level Architecture

The application follows a **layered architecture** with clear separation of responsibilities.

1. Presentation Layer (GUI)

This layer handles interaction between the user and the system.

- Displays windows, forms, buttons, and text areas.
- Accepts diary text and date input.
- Shows saved entries and status messages.
- Can be implemented using a C++ GUI framework such as **Qt** or **wxWidgets**.

2. Application/Control Layer

This layer coordinates user actions and system processes.

- Receives commands from the GUI.
- Controls workflows like create, save, edit, delete, and view entry.
- Connects GUI events to diary operations.
- Ensures correct processing order.

3. Business Logic Layer

This layer contains the core diary functionality.

- Creates new diary entries.
- Updates existing entries.
- Searches entries by date.
- Formats and validates diary content.
- Maintains the diary entry model with fields such as `date`, `title` (optional), and `content`.

4. Data Access Layer

This layer manages storage and retrieval of diary data.

- Saves entries into files or a lightweight local database.
- Loads entries when requested by the user.
- Maintains persistent storage across application sessions.
- Can use **text files, JSON, XML, or SQLite** depending on project scope.

5. Validation and Security Layer

This layer improves reliability and basic privacy.

- Checks for empty diary entries.
- Validates date format.

- Prevents data corruption during save operations.
- Can be extended later to include password protection or encryption.

Module Design

| Module | Responsibility |

|---|---|

| `main.cpp` | Application entry point and initialization |

| `MainWindow` | Main GUI screen for diary operations |

| `DiaryEntry` | Represents one diary record with date and content |

| `DiaryManager` | Handles add, edit, delete, search, and list operations |

| `FileHandler` / `DatabaseHandler` | Saves and loads diary data |

| `Validator` | Validates entry text and date input |

Data Flow

1. User opens the application and selects an action from the GUI.
2. The GUI sends the request to the control layer.
3. The control layer calls the diary manager.
4. The diary manager validates and processes the entry.
5. The data access layer stores or retrieves the diary record.
6. The result is returned to the GUI and displayed to the user.

Non-Functional Considerations

- **Usability**: simple and user-friendly interface.
- **Maintainability**: modular design supports code updates.
- **Scalability**: can later support search filters, categories, and password login.
- **Reliability**: validation and structured storage.

Algorithm:

Algorithm

Step 1: Start the application.

Step 2: Initialize the GUI window.

Step 3: Create required GUI components:

- Date label and date input field.
- Diary text area.
- Save button.
- View button.
- Clear button.
- Exit button.

Step 4: Wait for the user to enter the date and diary text.

Step 5: When the user clicks ****Save****:

- Read the date from the date field.
- Read the diary content from the text area.
- Check whether the date field is empty.
- Check whether the diary text area is empty.
- If any field is empty, display a warning message and stop the save process.
- Otherwise, open the diary file in append mode using C++ file handling.
- Write the date into the file.
- Write the diary entry below the date.
- Write a separator line to distinguish entries.
- Close the file.
- Show a success message in the GUI.

Step 6: When the user clicks ****View****:

- Open the diary file in read mode.
- If the file does not exist or cannot be opened, display an error message.

- Otherwise, read all saved entries line by line.
- Display the stored entries in a dialog box or text display area.
- Close the file.

Step 7: When the user clicks ****Clear****:

- Clear the date field.
- Clear the diary text area.

Step 8: When the user clicks ****Exit****:

- Close the application.

Step 9: Stop.

Conclusion:

The Personal Diary Application is a useful and practical C++ project that demonstrates how a simple GUI-based system can help users record daily entries with dates in an organized way. It combines a user-friendly interface with core programming concepts such as event handling, file storage, data validation, and modular design. By allowing users to create, save, view, and manage diary entries, the application provides a real-world solution for keeping personal notes securely and conveniently.

This project also shows the importance of separating the graphical interface from the logic and storage components, which makes the application easier to maintain and expand. The use of C++ for implementation helps build a strong foundation in object-oriented programming and desktop application development. Overall, the Personal Diary Application is a meaningful project for learning, practice, and future enhancement, with possible extensions such as password protection, search functionality, and database support.